

EC8207 – APPLIED BIG DATA ENGINEERING

MINI PROJECT ASSESSMENT

FinTech Fraud Detection Pipeline

EG/2020/3798	-	Abesundara W.H.S.
EG/2020/3975	-	Iyenshi A.U.T.
EG/2020/4136	-	Rajapaksha R.P.M.R.

1 INTRODUCTION

This project implements a comprehensive FinTech fraud detection pipeline following a **Lambda architecture** pattern that combines real-time stream processing with scheduled batch analytics. The system detects fraudulent credit card transactions in real-time while maintaining accurate reconciliation of transaction volumes through batch workflows.

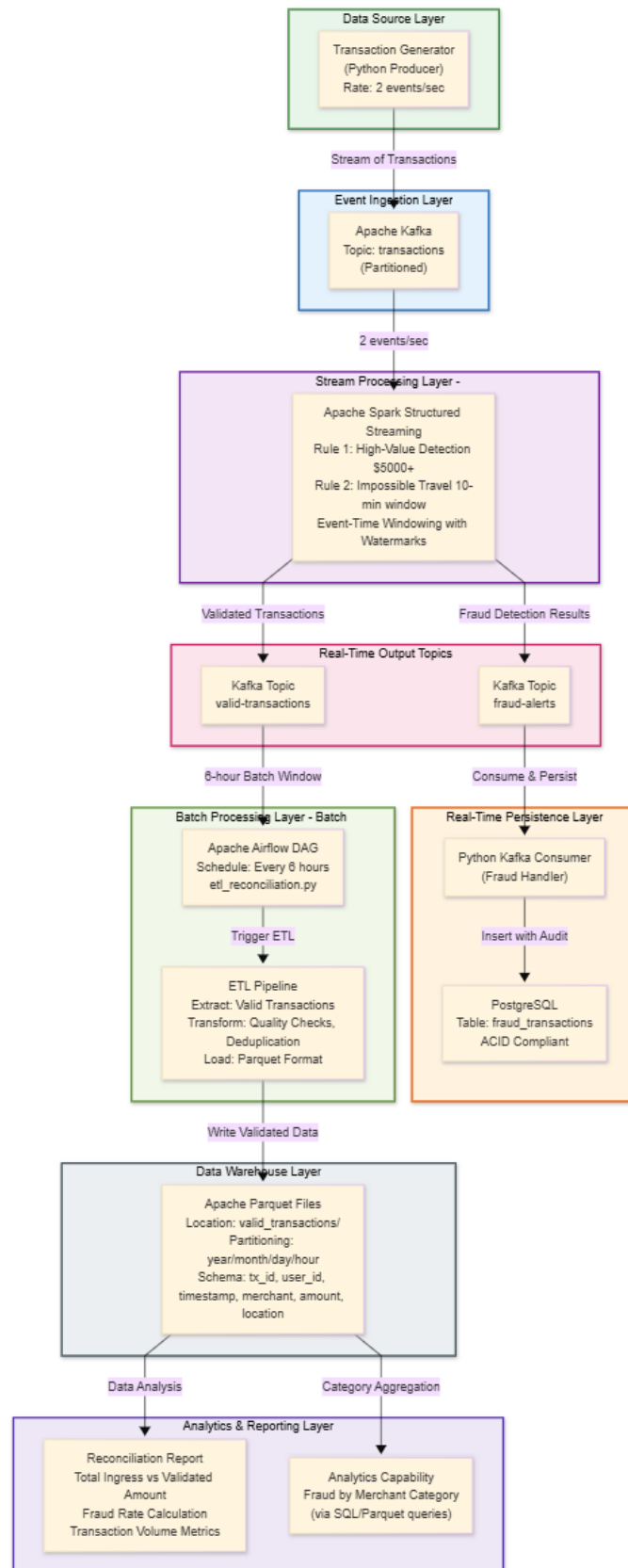
The pipeline architecture includes:

- **Ingestion Layer:** Python transaction generators producing realistic credit card transaction data
- **Streaming Layer:** Apache Kafka for event distribution; Apache Spark Structured Streaming for real-time fraud detection with deterministic rules (impossible travel and high-value detection)
- **Real-Time Alerts:** Immediate flagging of fraudulent transactions to PostgreSQL for investigation
- **Batch Processing Layer:** Apache Airflow orchestrating scheduled ETL jobs every 6 hours to validate and warehouse transactions
- **Analytics Layer:** Parquet-based data warehouse with reconciliation reports and merchant category analysis

This report documents the complete architecture, justification for technology choices, event-time handling semantics, batch orchestration logic, and ethical data governance considerations required for a production-grade fraud detection system.

2 SYSTEM OVERVIEW & ARCHITECTURE

The fraud detection pipeline implements a **Lambda architecture** combining real-time and batch processing paths:



Architecture Components:

- **Data Source Layer:** Transaction generator simulating a digital wallet provider, producing credit card transaction records with fields: `user\_id`, `timestamp`, `merchant\_category`, `amount`, `location`.
- **Event Ingestion Layer:** Apache Kafka distributes incoming transactions across partitioned topics, enabling horizontal scaling and consumer group parallelism.
- **Stream Processing Layer (Speed Path):** Apache Spark Structured Streaming applies two deterministic fraud detection rules with event-time awareness and watermarking (30-minute lateness tolerance). Sub-second latency enables immediate fraud response.
- **Real-Time Output & Persistence:** Fraudulent transactions are immediately written to PostgreSQL for investigation. Validated transactions are written to a separate Kafka topic for batch consumption.
- **Batch Processing Layer (Batch Path):** Apache Airflow orchestrates ETL jobs every 6 hours, consuming validated transactions from Kafka, performing quality validation and deduplication, and loading to the data warehouse.
- **Data Warehouse Layer:** Apache Parquet provides columnar storage with temporal partitioning (year/month/day/hour), enabling efficient analytical queries and data retention management.
- **Analytics & Reporting Layer:** Scheduled reports generate reconciliation comparisons (total ingress vs. validated amounts) and merchant category fraud analysis for business insights.

3 TOOLS & TECHNOLOGIES JUSTIFICATION

- **Apache Kafka:** Enables horizontal scaling for high-throughput transaction processing. Topic partitioning distributes load; message persistence provides replay capability for auditing and regulatory compliance (immutable event logs).
- **Apache Spark Structured Streaming:** Unifies batch and stream processing with identical logic, ensuring consistency between real-time alerts and batch reconciliation. Micro-batch processing provides exactly-once semantics essential for financial accuracy. Windowing enables complex temporal queries (e.g., "impossible travel within 10 minutes").
- **Apache Airflow:** Orchestrates 6-hour ETL jobs with DAG-based dependency management. Audit logging and retry logic satisfy compliance requirements and handle transient failures.
- **Apache Parquet:** Columnar compression improves analytical query performance 10-100x compared to row-oriented storage. Schema evolution supports adding transaction fields without rewriting historical data.
- **PostgreSQL:** Provides ACID guarantees for fraud transaction records, preventing data loss and corruption. Supports immediate investigation access.

- **Docker Compose:** Ensures consistent software versions across environments, eliminating deployment inconsistencies.

4 REAL-TIME STREAM PROCESSING LOGIC

Two deterministic fraud rules operate with event-time awareness and sub-second latency:

- **Rule 1 - High-Value Detection:** Transactions exceeding \$5,000 trigger immediate fraud alerts.
- **Rule 2 - Impossible Travel:** Multiple transactions from different countries within 10 minutes flag both as fraudulent, detecting stolen credentials.

```
fraud_transactions = transactions.filter(  
    (col("amount") > 5000) |  
    ((approx_count_distinct("location") > 1) &  
    (window(col("event_time"), "10 minutes").isNotNull()))  
)
```

Both rules prioritize **latency over batch efficiency**. Frauds are output immediately to Kafka, enabling downstream blocking systems to respond within seconds rather than hours.

5 EVENT TIME VS. PROCESSING TIME

- **Event Time:** When a transaction occurred (timestamp field)
- **Processing Time:** When Spark receives the message (after Kafka delivery delays)

This distinction is critical for fraud detection. Example: A fraudster initiates Singapore and London transactions at 14:00 and 14:05 UTC respectively. Network delays cause Singapore to arrive at Spark at 14:20 UTC while London arrives at 14:08 UTC.

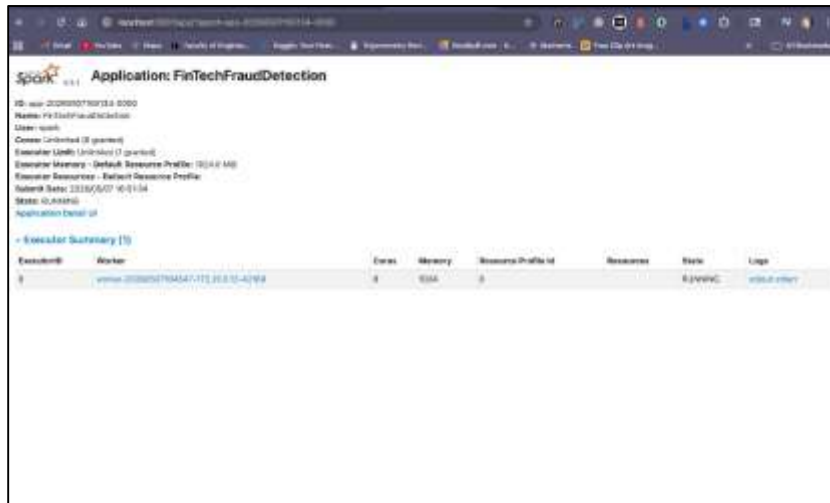
Using **processing time** would measure 12 minutes between transactions, missing impossible travel. Using **event time** correctly identifies 5 minutes, triggering fraud alerts.

Implementation:

```
transactions = transactions.withColumn("event_time", to_timestamp(col("data.timestamp")))

transactions.withWatermark("event_time", "30 minutes").groupBy(col("user_id"),  
window(col("event_time"), "10 minutes"))
```

With event-time semantics, fraud detection is deterministic and auditable, independent of message arrival order.



6 BATCH PROCESSING & ORCHESTRATION

Scheduled batch jobs every 6 hours validate and warehouse transaction data. The Airflow DAG (`infrastructure/airflow/dags/etl\_reconciliation.py`) pipeline:

1. **Extract:** Validated transactions from PostgreSQL or `valid-transactions` Kafka topic
2. **Transform:** Quality validation, deduplication, timestamp normalization
3. **Load:** Write to Parquet partitioned by `year/month/day/hour`
4. **Reconcile:** Generate reports comparing total ingress (fraud + valid) vs. validated amounts

Data Warehouse Schema:

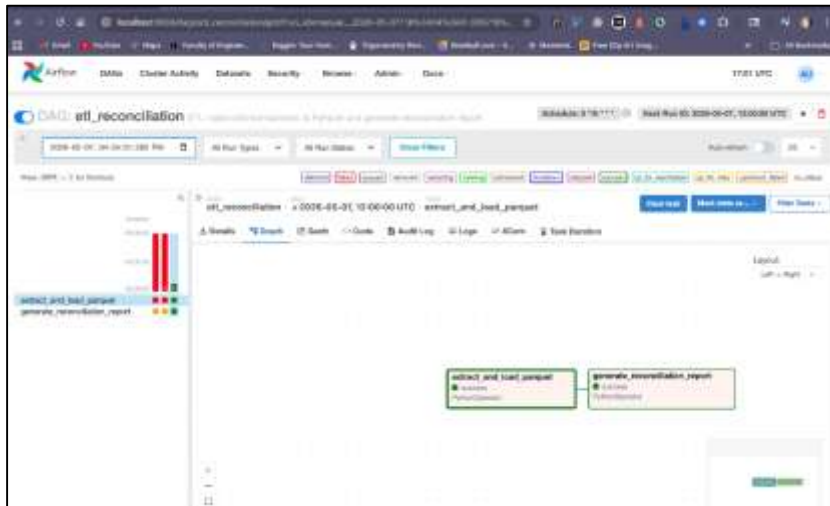
valid_transactions/year=2026/month=05/day=07/hour=16/data.parquet

Fields: `transaction\_id`, `user\_id`, `event\_time`, `merchant\_category`, `amount`, `location`

Reconciliation Metrics:

- Total Ingress: Sum of all transactions received
- Validated Amount: Sum of non-fraud transactions
- Fraud Loss: Difference (total - validated)
- Fraud Rate: Loss / total ingress (percentage)

A human-readable reconciliation report is generated showing these metrics and a balanced verification (Valid + Fraud = Total Ingress).



7 ANALYTICS & REPORTING OUTPUT

The pipeline generates analytics outputs at both real-time and batch intervals:

Real-Time Alerts

- Fraud detection events emitted to `fraud-alerts` Kafka topic with fraud type and reasoning
- PostgreSQL stores detailed fraud transaction records for immediate investigation
- Console output enables operational visibility during development

Batch Reconciliation Report

The scheduled batch job generates a reconciliation report (`infrastructure/data\_warehouse/reports/reconciliation\_YYYY-MM-DD\_HH-MM-SS.txt`) containing:

- Time Window: Period covered (e.g., 2026-05-07 16:00 to 22:00)
- Total Ingress Amount: Sum of all transactions received
- Validated Amount: Sum of non-fraud transactions
- Fraud Loss: Difference (total - validated)
- Fraud Rate: Fraud loss / total ingress (percentage)
- Transaction Counts: Total received, total flagged, validation rate

Merchant Category Analysis Capability

Fraud transaction data stored in PostgreSQL and Parquet warehouse includes `merchant\_category` field, enabling downstream analytical queries to aggregate fraud attempts by industry (e.g., Travel, Retail, Electronics, Gaming, Pharmacy). This data can be analyzed via SQL queries on the `fraud\_transactions` table or Parquet files for identifying high-risk merchant categories.

8 ETHICS & DATA GOVERNANCE

Fraud detection requires sensitive financial data with privacy and ethical considerations.

- **Privacy Risks:** Transaction monitoring exposes spending habits and locations. Unauthorized access enables targeted crimes. Merchant data infers sensitive information (pharmacy visits → health conditions, political donations → beliefs). False positives block legitimate transactions, harming customer trust.
- **Data Governance Safeguards:**
 - **Purpose Limitation:** Restrict data to fraud detection only; separate teams prevent secondary uses
 - **Data Minimization:** Store only ID, amount, merchant, location, timestamp; delete unnecessary PII after 90 days
 - **Encryption:** TLS 1.3 in transit, AES-256 at rest; rotate keys every 90 days
 - **Access Control:** Role-based access—fraud team has database access; analytics team does not
 - **Retention Limits:** Delete records after 180 days unless under investigation; maintain audit logs 7 years
 - **Transparency:** Inform users of monitoring; allow fraud flag appeals
- **Regulatory Compliance:**
 - **GDPR:** Lawful basis (fraud prevention), honor deletion/portability rights
 - **PCI DSS:** Encrypt payment info, restrict access, maintain audit logs
 - **Fair Lending:** Monitor fraud rates across demographics for algorithmic bias

9 CONCLUSION

This FinTech fraud detection pipeline implements a complete Lambda architecture for real-time fraud detection combined with comprehensive batch analytics. The system handles event-time semantics explicitly, ensuring deterministic fraud detection results despite network delays. The streaming path (Kafka → Spark → PostgreSQL) achieves sub-second latency for immediate fraud blocking, while the batch path (Kafka → Airflow ETL → Parquet warehouse) provides auditable reconciliation reports and merchant category analytics.

By separating speed and batch layers, the architecture balances operational requirements (fast fraud response) with analytical needs (comprehensive transaction auditing and reporting). Proper data governance safeguards such as encryption, retention policies, access controls, and transparency which ensure ethical use of financial data while maintaining regulatory compliance.